PROJECT DOCUMENTATION

AI Travel Planner

Streamlit · LangChain · Gemini AI

Table of Contents

1. Project Introduction

AI Travel Planner is a web application that automatically generates a complete, personalized travel plan based on the user's inputs. Instead of browsing multiple websites for flights, hotels, places, and weather, the user simply enters their travel details and the AI handles everything — delivering a full plan in seconds.

The application is powered by Google Gemini AI for intelligent content generation, LangChain for orchestrating multiple AI tools, and Streamlit for the interactive web interface. The project is deployed live on Streamlit Community Cloud and version-controlled via GitHub.

> **Live Application:** https://travelplanner-zsnfaaygdkxyao9qkxwz35.streamlit.app/
> **Repository: https://github.com/Aryan001521/Travel_planner**

2. Objective

The goal of this project is to eliminate the effort of manual travel research by delivering a complete AI-generated travel plan — including flights, hotels, places, weather, itinerary, and budget — through one clean interface.

- Build an AI-powered travel planner that generates real, usable trip plans.
- Use LangChain agents to intelligently call specialized tools for each travel need.
- Provide both budget and luxury trip planning modes.
- Validate user inputs and handle errors gracefully.
- Deploy a production-ready app accessible from any browser.

3. Technologies Used

Technology	Version	Purpose
Python	3.10+	Core programming language
Streamlit	1.32+	Web application frontend
LangChain	0.1+	AI agent orchestration framework
Google Gemini AI	1.5 Pro	Large language model for AI generation
google-generativeai	0.5+	Python SDK for Gemini API
python-dotenv	1.0+	Environment variable management
GitHub	—	Version control and source hosting
Streamlit Cloud	—	Free cloud deployment platform

4. Project Features

Feature	Description
Flight Suggestions	AI generates 3–5 flight options with airline, timing, and estimated price.
Hotel Recommendations	Recommends hotels based on trip type (budget/luxury) with ratings and price range.
Tourist Places	Lists top attractions at the destination with short descriptions.
Weather Details	Provides a weather summary for the travel period including temperature and clothing tips.
Day-wise Itinerary	Creates a structured day-by-day plan covering morning, afternoon, and evening.
Estimated Budget	Breaks down total estimated cost by flights, hotels, food, transport, and activities.
Invalid City Validation	Detects and rejects invalid or non-existent city names with an error message.
Dark UI	Clean, modern dark-themed interface built with custom CSS in Streamlit.
Trip Type Selection	Users can choose between Budget and Luxury modes which affect all AI outputs.
Responsive Design	Works across desktop and mobile browsers.

5. Working of the Project

The application works in a straightforward sequence: the user fills in the travel form, the LangChain agent processes the request, each AI tool generates its section, and the complete plan is displayed on screen.

Step-by-Step Flow

1. User opens the app and fills in: Source City, Destination City, Number of Days, Trip Type (Budget / Luxury).
2. On clicking Generate Plan, the inputs are validated. If a city is invalid, an error is shown immediately.
3. The TravelAgent in LangChain receives the inputs and runs the AI tools in sequence.
4. Each tool (Flight, Hotel, Weather, Places, Budget, Itinerary) sends a structured prompt to Gemini AI and gets back formatted data.
5. All outputs are assembled and displayed in organized sections on the Streamlit interface.

> **Total Response Time:** Approximately 15–30 seconds for a complete plan
> **AI Model:** Google Gemini 1.5 Pro via LangChain ZERO_SHOT_REACT_DESCRIPTION agent

6. Module Explanation

main.py — Streamlit Frontend

Entry point of the application. Renders the UI, collects user inputs, triggers the agent, and displays all results. Contains the custom dark theme CSS and output formatting logic.

agent/travel_agent.py — LangChain Agent

Initializes the Gemini LLM and registers all six tools with LangChain. Runs the ZERO_SHOT_REACT_DESCRIPTION agent which automatically decides which tools to call and in what order based on the user query.

tools/flight_tool.py — Flight Module

Generates 3–5 flight suggestions with airline name, departure/arrival time, duration, and estimated price. Budget and Luxury modes return different price ranges.

tools/hotel_tool.py — Hotel Module

Recommends hotels at the destination with name, star rating, price per night, and location. Adjusts recommendations based on the selected trip type.

tools/weather_tool.py — Weather Module

Returns a weather summary for the destination during the travel period, including temperature range, precipitation likelihood, and recommended clothing.

tools/place_tool.py — Places Module

Lists the top tourist attractions at the destination with a short description of each place and the best time to visit.

tools/budget_tool.py — Budget Module

Estimates total travel cost broken down by category: Flights, Hotels, Food, Local Transport, and Activities. The breakdown differs between Budget and Luxury modes.

tools/itinerary_tool.py — Itinerary Module

Generates a day-wise travel plan for the number of days selected. Each day is split into Morning, Afternoon, and Evening segments with specific activities and restaurant suggestions.

tools/display_tool.py — Display Utility

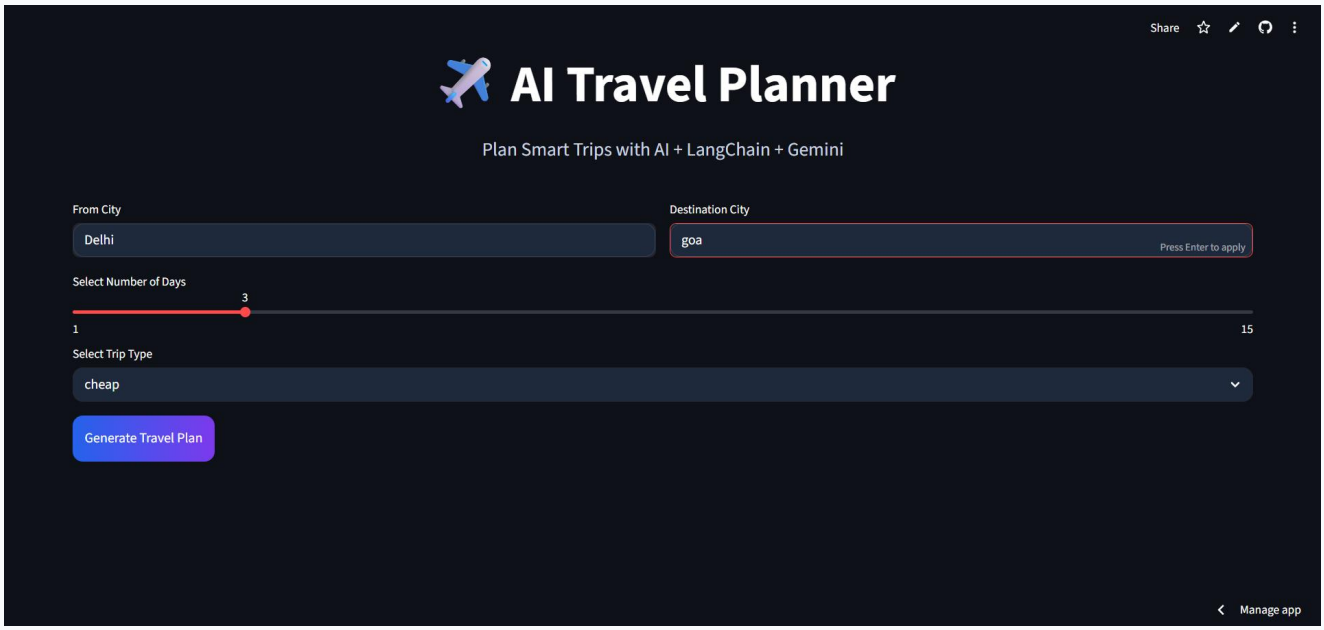
Helper functions that convert raw AI-generated JSON data into formatted HTML/Markdown for clean display in the Streamlit interface.

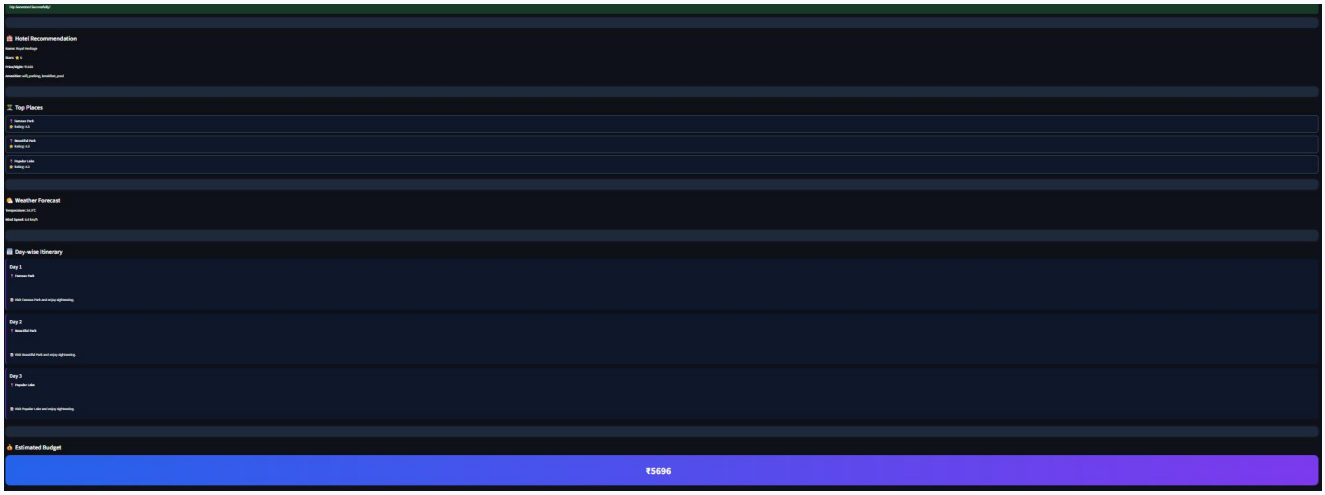
7. UI Design

The frontend is built with Streamlit and styled using custom CSS injected via st.markdown(). The interface uses a dark theme throughout for a modern, professional look.

Design Highlights

- Dark background (#0E1117) with high-contrast white text.
- Teal/blue accent colors for buttons, borders, and section highlights.
- Two-column layout for the input form (source and destination side by side).
- Expandable cards or structured sections for each output category.
- Loading spinner displayed while the AI generates the plan.
- Responsive layout that adapts to mobile screen widths.

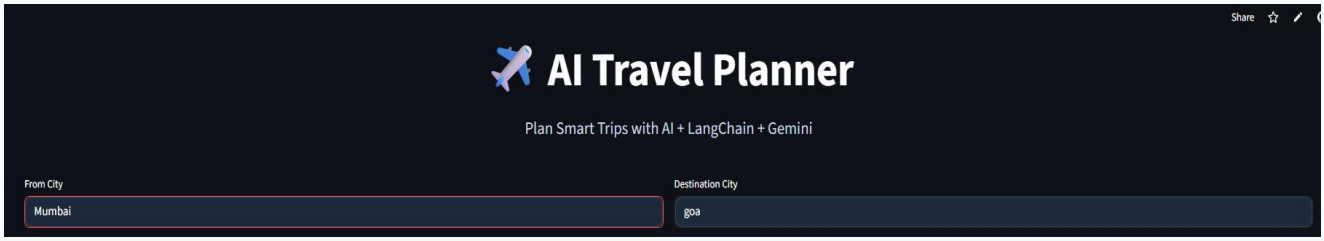



Hotel Recommendation
Top Places
Weather Forecast
Day-wise Itinerary
Day 1
Day 2
Day 3
Estimated Budget
₹5696

8. Input and Output

User Inputs

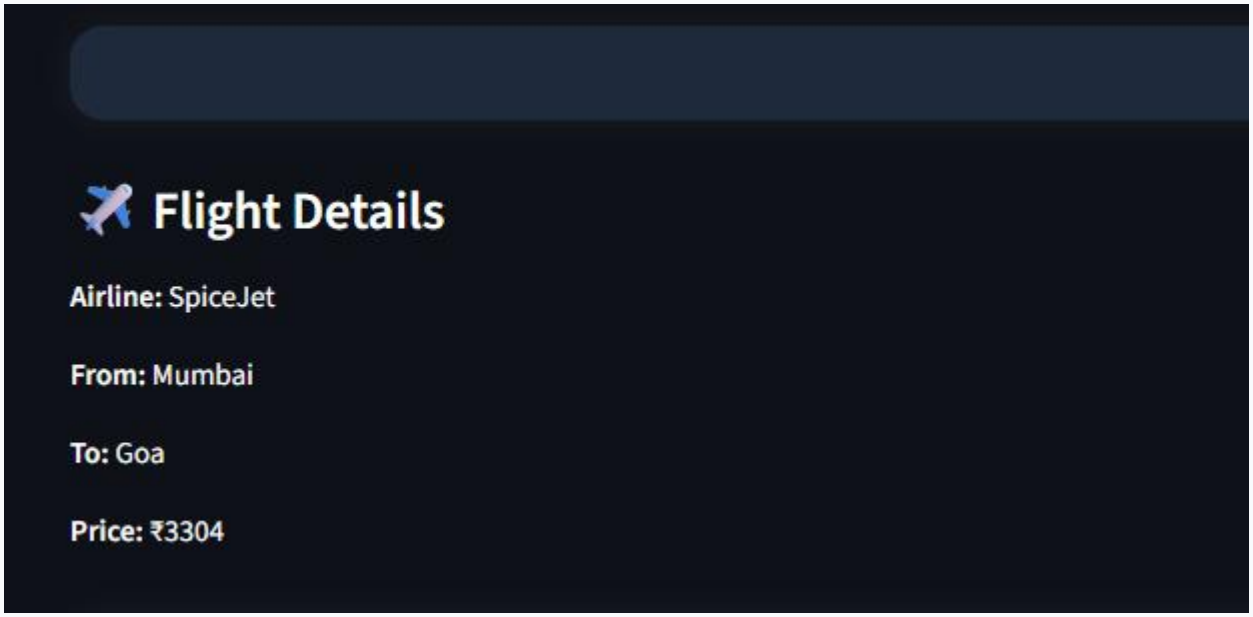
Input Field	Type	Example
Source City	Text	Mumbai
Destination City	Text	Goa
Number of Days	Slider (1–30)	7
Trip Type	Radio Button	Budget / Luxury



Generated Outputs

Flight Suggestions

AI returns 3–5 flight options with airline, timings, duration, and price. Prices are adjusted based on Budget or Luxury mode.



Hotel Recommendations

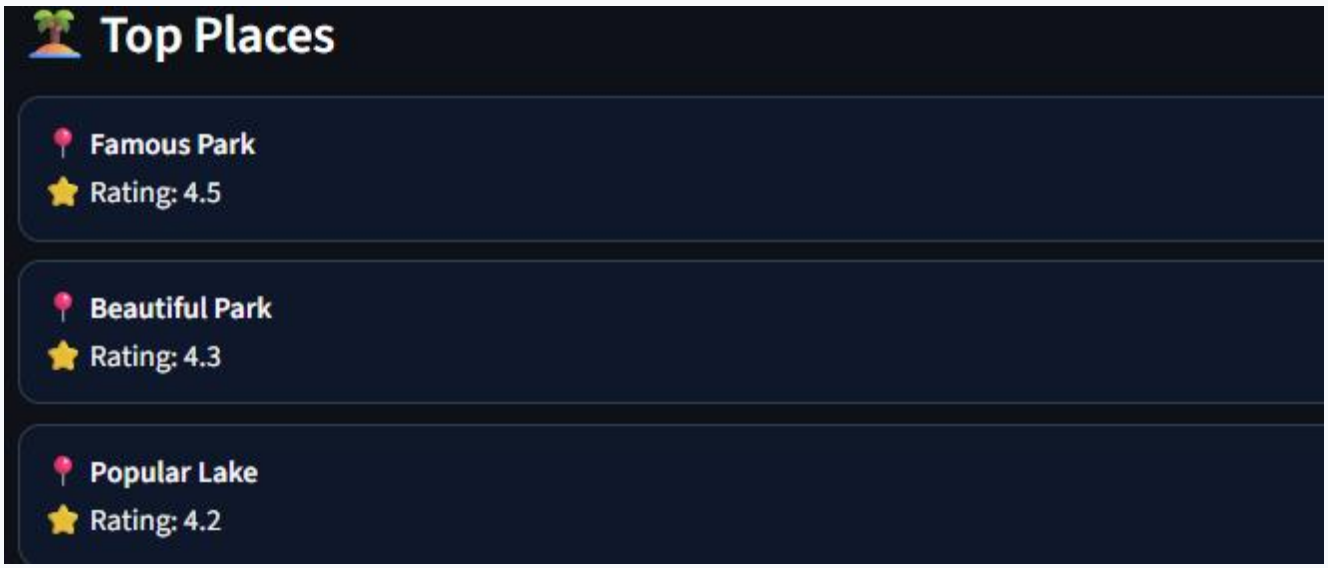
Returns 4 hotel options with star rating, price per night, and location within the city.

Weather Details

A short weather summary covering temperature, rain probability, and what to pack.

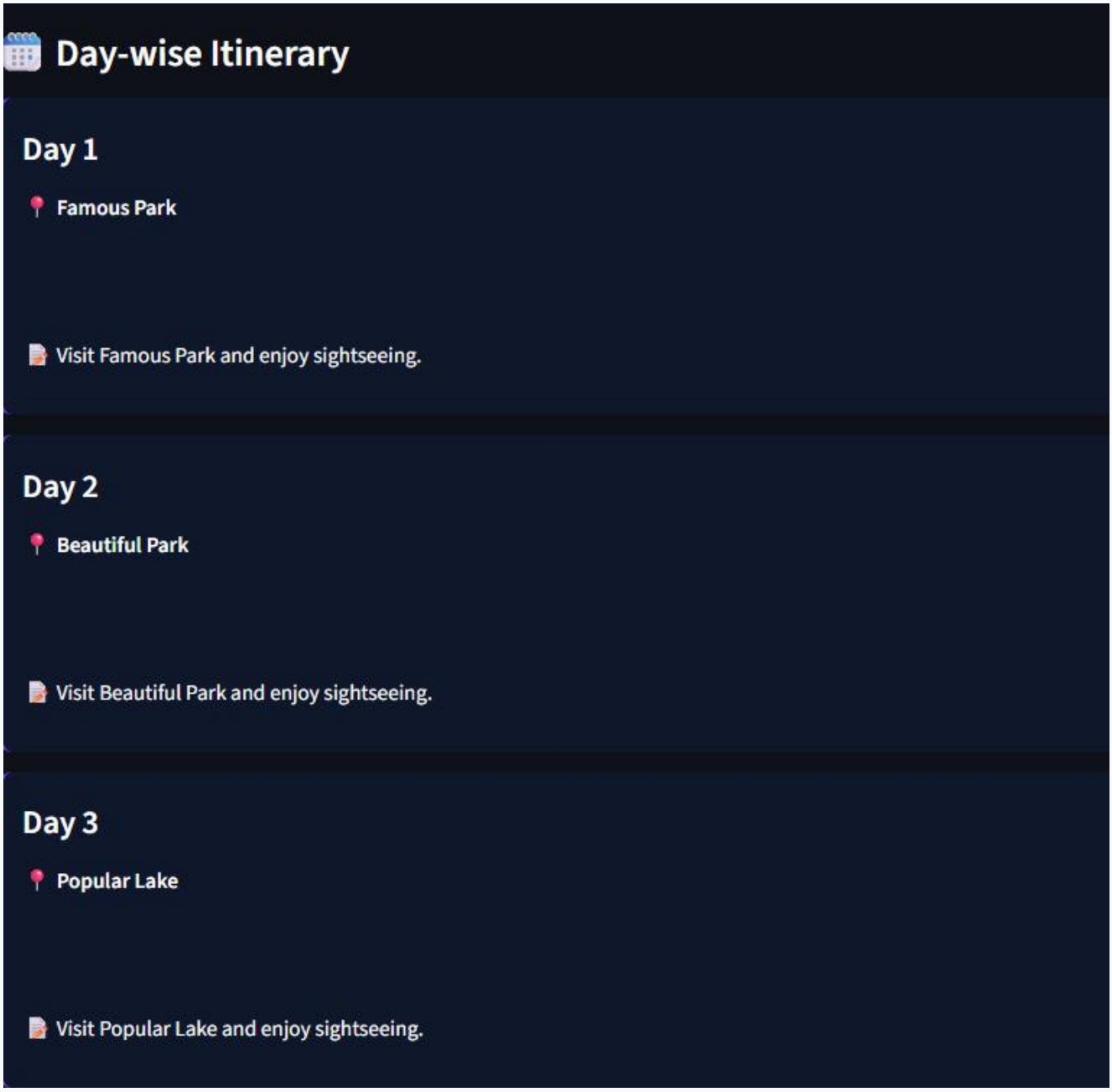
Tourist Places

A curated list of top attractions with descriptions.



Day-wise Itinerary

A complete day-by-day plan with morning, afternoon, and evening activities for each day of the trip.



Budget Breakdown

Estimated cost breakdown by category with a grand total in INR.

Invalid City Error

When a user enters an invalid city name, the system shows a clear error message and does not proceed.

9. Code Workflow

LLM Initialization

The Gemini model is initialized using LangChain's ChatGoogleGenerativeAI wrapper with temperature set to 0.7 for balanced, creative yet accurate responses.

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model='gemini-1.5-pro',
    temperature=0.7,
    convert_system_message_to_human=True
)
```

Tool Definition

Each module is registered as a LangChain tool using the @tool decorator. The function docstring acts as the tool description that the agent uses to decide when to call it.

```
from langchain.tools import tool

@tool
def get_flights(query: str) -> str:
    '''Get flight suggestions between two cities for a trip.'''
    prompt = build_flight_prompt(query)
    response = gemini_model.generate_content(prompt)
    return safe_parse_json(response.text)
```

Agent Setup

The agent is initialized with all tools and the Gemini LLM. The ZERO_SHOT_REACT_DESCRIPTION agent type uses the ReAct loop — it thinks, picks a tool, observes the result, then repeats until the plan is complete.

```
from langchain.agents import initialize_agent, AgentType

agent = initialize_agent(
    tools=[get_flights, get_hotels, get_weather,
           get_places, get_budget, get_itinerary],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    handle_parsing_errors=True,
```

```
    max_iterations=10,
    verbose=True
)
```

Streamlit Trigger

```
if st.button('Generate Travel Plan'):
    with st.spinner('AI is planning your trip...'):
        agent = TravelAgent()
        result = agent.plan(source, dest, days, trip_type)
    display_results(result)
```

10. Error Handling

Error Type	How It Is Handled
Empty city input	UI-level check shows an error before the agent is called.
Invalid city name	Regex validation blocks non-alphabetic characters. Gemini also validates contextually.
Same source and destination	Caught at the UI level with a descriptive error message.
Gemini API failure	try/except around all API calls. Fallback error message is shown to the user.
Malformed JSON from Gemini	safe_parse_json() strips markdown fences and falls back to raw text if parsing fails.
API quota exceeded	Error is caught and a user-friendly message is displayed.
Network timeout	Wrapped in exception handler; agent terminates gracefully with a message.

```
def safe_parse_json(raw: str) -> dict:
    try:
        clean = raw.strip().lstrip('```json').rstrip('```').strip()
        return json.loads(clean)
    except json.JSONDecodeError:
        return {'raw_response': raw, 'error': True}
```

11. Deployment Process

Local Setup

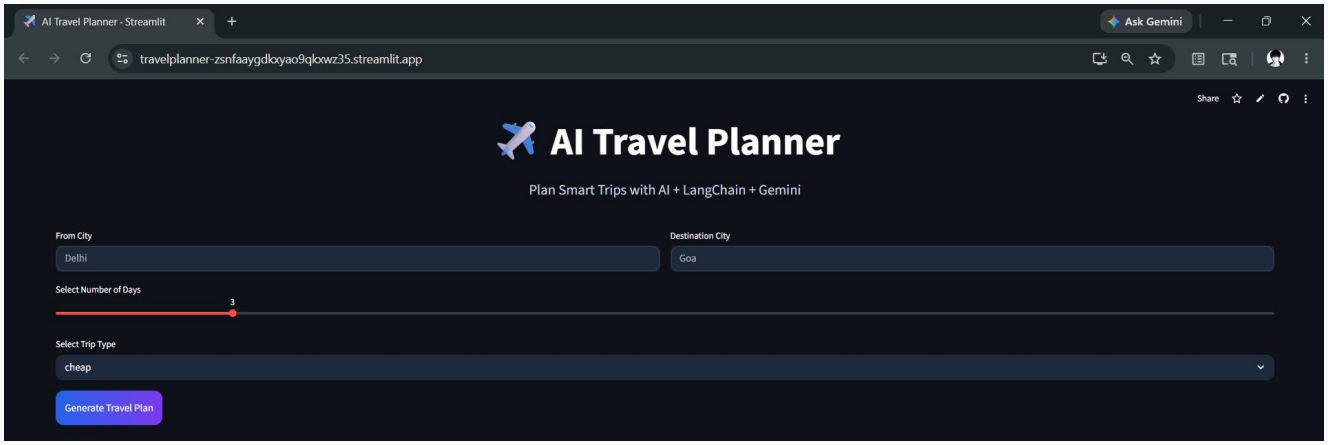
```
# 1. Clone the repository
git clone https://github.com/[username]/ai-travel-planner.git
cd ai-travel-planner

# 2. Install dependencies
pip install -r requirements.txt

# 3. Add your Gemini API key
echo 'GEMINI_API_KEY=your-key-here' > .env

# 4. Run the app
streamlit run main.py
```

Streamlit Cloud Deployment

6. Push the project to a GitHub repository.
7. Go to share.streamlit.io and sign in with GitHub.
8. Click New App, select the repository, and set main.py as the entry point.
9. Go to Advanced Settings → Secrets and add:

```
GEMINI_API_KEY = "your-gemini-api-key"
```

10. Click Deploy. The app goes live at a public Streamlit URL within 2–5 minutes.
11. Any push to the main branch automatically triggers a redeployment.



Auto-deploy: Every git push to main triggers automatic redeployment on Streamlit Cloud.

12. Challenges Faced

Challenge	How It Was Resolved
Inconsistent JSON from Gemini	Wrote a robust safe_parse_json() parser that strips markdown fences and handles fallbacks.
Slow sequential tool calls	Accepted as a current limitation; future plan is to parallelize independent tool calls using asyncio.
LangChain–Gemini compatibility	Used langchain-google-genai package with convert_system_message_to_human=True for compatibility.
API key security on deployment	Used Streamlit Secrets Management instead of hardcoding keys; .env excluded via .gitignore.
Prompt tuning for structured output	Iterated on prompts using role prompting, JSON schema hints, and explicit field-level instructions.

13. Future Improvements

Improvement	Description
Real-time Flight Data	Integrate Amadeus or Skyscanner API for live flight pricing and availability.
Live Hotel Booking	Connect Booking.com or Expedia API for real hotel availability.
Live Weather API	Replace AI weather summary with real-time data from OpenWeatherMap.
Parallel Tool Execution	Run independent tools (flights, hotels, weather, places) concurrently using asyncio to cut response time by ~60%.
PDF Export	Allow users to download the generated travel plan as a formatted PDF.
Multi-city Trips	Support itineraries with multiple stops (e.g., Delhi → Paris → Amsterdam).
Conversational Mode	Add a chat interface so users can refine their plan through natural follow-up questions.
User Accounts	Save and revisit past travel plans using Firebase or Supabase authentication.

14. Conclusion

AI Travel Planner successfully demonstrates how modern AI tools — Gemini, LangChain, and Streamlit — can be combined to solve a real and practical problem. The application takes a task that normally takes hours of manual research and completes it in under a minute, delivering flights, hotels, weather, places, a day-wise itinerary, and a budget estimate through one clean interface.

The modular architecture makes it easy to extend — any tool can be upgraded to use a real API without changing the rest of the system. The project is fully deployed and publicly accessible, making it a strong demonstration of end-to-end AI application development from design to deployment.

What Was Built	Key Skills Demonstrated
End-to-end AI travel planning app	LangChain agent + tool architecture
6 specialized AI tool modules	Prompt engineering with Gemini AI
Dark-themed Streamlit UI	Streamlit frontend development
Input validation & error handling	Robust error handling patterns
Live cloud deployment	GitHub + Streamlit Cloud CI/CD

Prepared by **[ARYAN SHARMA]** · [aryansharmabf85@gmail.com]
GitHub: https://github.com/Aryan001521/Travel_planner